

System 13: High-Throughput Logging Framework

Zero-Prerequisite Intuition: The "Airport Black Box" Metaphor What is a logging framework, and why can't we just use `print()`? Imagine an airplane in mid-flight. Hundreds of sensors (engine temperature, altitude, hydraulic pressure) record telemetry every millisecond. If the pilot had to stop flying the airplane and personally hand-write every temperature reading on paper before pulling the steering wheel, the plane would crash!

Instead, sensors dump readings into an automated **Black Box** in the background while the pilot continues flying uninterrupted.

In high-scale web servers, every request generates logs. If an application thread blocks on a synchronous disk write (`file.write()`) on every request, your server throughput drops from 10,000 requests/sec to 100 requests/sec because hard drives and networks are 100,000x slower than CPU RAM!

The **#1 Low-Level Design question at Uber, Stripe, and Google** is: *"Design a multi-threaded, asynchronous, high-throughput logging library that supports log levels, custom formatters, multiple destinations (Console, File, S3), and never blocks worker threads."*

1. Requirements & System Boundaries

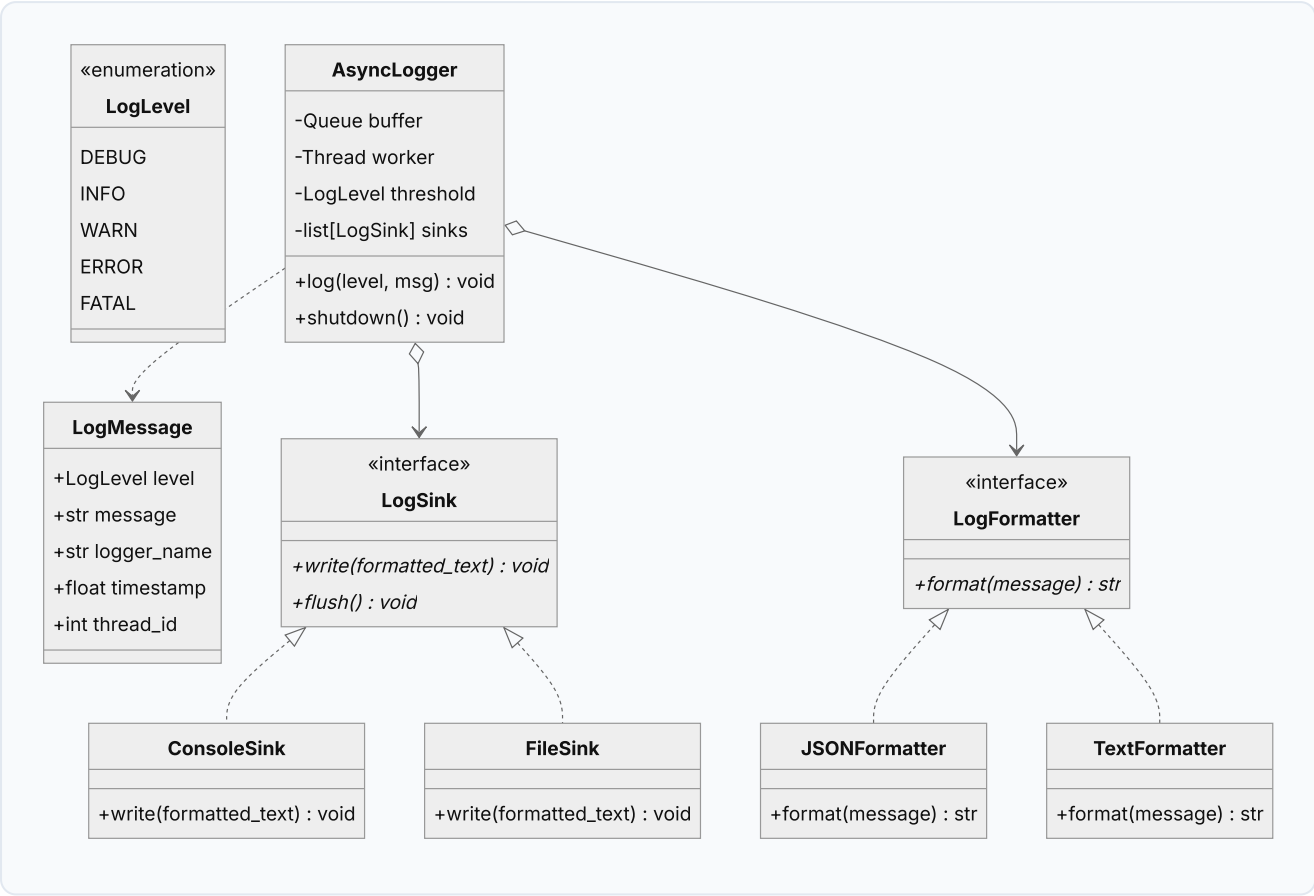
Functional Requirements

1. **Log Levels:** Strict priority filtering: `DEBUG (10) < INFO (20) < WARN (30) < ERROR (40) < FATAL (50)`.
2. **Pluggable Appenders / Sinks:** Support writing to multiple destinations simultaneously (Console, File, Remote Network).
3. **Pluggable Formatters:** Support converting log records into Human-Readable Plain Text or Machine-Readable JSON.

Non-Functional Requirements

- **Zero Latency Overhead (Asynchronous):** Worker threads log in $< 1\mu s$ by pushing into an in-memory queue. A background thread handles batch flushing to disk.
- **Buffer Overflow Policy:** What happens when logs arrive faster than disk can write? (Drop lowest priority logs vs. block producer).

2. Design Patterns Architecture



3. Complete Python Implementation

4. Verification & Stress-Testing

4. Edge Cases, Tests and Extensions

Behaviour under pressure

Situation	What happens	Verdict
Level below the threshold	Dropped before any allocation	Correct and cheap
Queue full	<code>put_nowait</code> raises <code>queue.Full</code> ; the record is dropped and a warning is printed	A deliberate overload policy: logging must never block the request path. Count the drops and expose the counter
A sink raises	The error is printed and the other sinks still receive the record	Isolation is right; add a circuit breaker so a dead sink is not retried every record
<code>shutdown()</code>	Sets the event, joins the worker for up to 2 seconds, then flushes sinks	A long backlog can outlive the 2 second join and lose records
<code>log()</code> after <code>shutdown()</code>	Record is queued and never emitted	Silent loss; reject or warn instead
Process exit without <code>shutdown()</code>	Worker is a daemon thread, so queued records vanish	Register <code>atexit(logger.shutdown)</code>
Ordering	Preserved per producer thread (FIFO queue), interleaved across threads	Good enough; do not promise a global order

Tests

This block extends the implementation above. `GateSink` blocks the worker so the overflow behaviour is deterministic.

Extensions interviewers ask for

1. **File sink with rotation:** rotate by size or time; write to a temporary name and rename, and keep `flush()` meaningful (`fsync` on shutdown).
2. **Batching:** drain up to N records per wake-up and write them in one call; this is how throughput climbs from thousands to hundreds of thousands of records per second.

3. **Structured context (request id, user id):** use `contextvars.ContextVar` so the id is attached automatically in every thread or task without passing it around.
4. **Dropped-record metric:** increment a counter in the `queue.Full` branch and log it once per second from the worker, so an overload is visible instead of silent.
5. **Multiprocess safety:** several processes writing one file need a single writer process or a socket handler; two processes appending with their own buffers will interleave lines.

Follow-up questions

- *Why a queue and a background thread rather than writing in the caller?* Disk and network latency would otherwise land on the request path. The queue decouples producer latency from sink latency at the cost of possible loss on a crash.
- *Block or drop when the buffer is full?* Dropping protects availability; blocking protects completeness. Choose per level: never drop `ERROR`, freely drop `DEBUG`.
- *How does Python's own `logging.handlers.QueueHandler` relate?* It is the same design: handlers enqueue records and a `QueueListener` thread owns the slow handlers.